Self-Driving Car Case - Appendix To Paper: Towards Better Adaptive Systems by Combining MAPE, Control Theory, and Machine Learning

Danny Weyns
KU Leuven, Belgium
Linnaeus University, Sweden
danny.weyns@kuleuven.be

Bradley Schmerl
Carnegie Mellon University
Pittsburgh, USA
schmerl@cs.cmu.edu

Masako Kishida
National Institute of Informatics
Tokyo, Japan
kishida@nii.ac.jp

Alberto Leva
Politecnico di Milano
Milan, Italy
alberto.leva@polimi.it

Marin Litoiu
York University
York, Canada
mlitoiu@yorku.ca

Necmiye Ozay
University of Michigan
Ann Arbor, MI, USA
necmiye@umich.edu

Colin Paterson
University of York
York, United Kingdom
colin.paterson@york.ac.uk

Kenji Tei
Waseda University
Tokyo, Japan
ktei@aoni.waseda.jp

I. Application

Consider a self-driving car with a goal to drive from its current location to a *target destination* taking into account user-preferences, such as *preferred arrival time* and *road preferences*. At a high-level, the car needs to plan a route, i.e., a sequence of *waypoints*. At a lower-level, the *throttle*, *braking* and *steering* commands need to be determined, while guaranteeing safety and performance.

The car should respect safety constraints at all time, such as never crash into another vehicle or object on the road, stop at red lights, etc. Performance requirements are softer and include passenger comfort and making progress towards the goal in a timely manner. To that end, the system is equipped with sensors that measure the vehicle *acceleration*, *jerk*, and *fuel level*, and radar and cameras that track its *relative position* and *vehicle distance* in its immediate environment.

The vehicle is subject to uncertainties. At global level, uncertainties are related to *traffic information*, such accidents and traffic jams. These properties affect the overall progress and timeliness of the trip. There are also uncertainties at local level related to the *local traffic context*, e.g., weather conditions, road surface, winds, and uncertainty about other road participants such as their actual positions, and sometimes velocities, etc. Some of the quantities of the different types of uncertainties can be measured at runtime, but often their effect needs to be inferred from the high dimensional data, for instance sensory data from radar and cameras.

II. Designing the Controller

To achieve its goals in the presence of uncertainties, we design the self-driving car navigation as an adaptive system that combines MAPE (Monitor-Analysis-Plan-Execute [1]–[7]) and CT (Control Theory [8]–[10]), supported by ML (Machine Learning, e.g., [11]–[13]). The architecture that comprises four layers (Goals, MAPE, Vehicle, CT controllers) is shown in Figure 1.

A. Goal layer

The goal layer takes the user goals, such as the target destination, preferred arrival time, and road preferences and where applicable translates these into technological goals for MAPE; e.g., road preferences are mapped to weights associated with types of streets/highways.

B. MAPE layer

The MAPE layer ensures the user goals and informs the user about the traffic plan and changes to it. An initial plan that consists of a sequence of road segments and waypoints is calculated. To that end, MAPE uses the user goals, the current traffic conditions, e.g. traffic delays, and real time data of the scene of the vehicle (e.g., traffic flow). MAPE also takes as input provided by low-level control about the position and velocity of the vehicle, jerk, and fuel level. The calculated waypoints are then communicated to the lower level control. The MAPE loop continuously monitors the vehicle status, the global and local traffic conditions in the environment, and adapt the plan when necessary.

C. CT layer

The CT layer comprises two subsequent controllers that control the vehicle. A performance controller generates paths based on the waypoints provided by the MAPE layer, taking into account nearby obstacles/vehicles, the acceleration and fuel level, to determine the vehicle's control operations while optimising some cost, e.g., fuel consumption. A safety controller tracks the vehicle's control operations and its local context ensuring safe and accurate driving. The performance controller can be realised using MPC, and the safety controller as a PID controller wrapped with a safety enforcing supervisor based on invariant sets or control barrier functions.

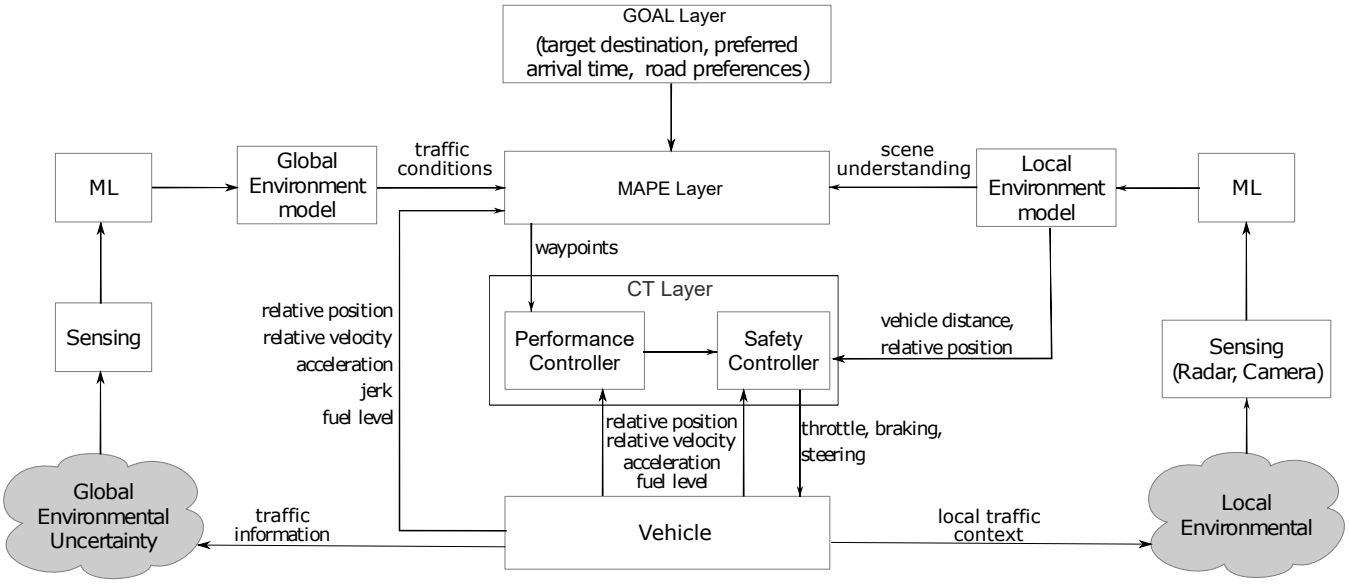

Fig. 1. Adaptive self-driving car system.

D. System layer

The vehicle that is subject to adaptation (we abstract away from internal control modules of the vehicle). The system operates in the environment that is subject to different types of uncertainties. As shown in the figure, we consider here both local and global types of uncertainties.

E. The role of ML

The MAPE layer can use ML for high-level planning, where a model is trained based on the time of the day, weather conditions, etc., to predict the traffic conditions in real time. ML can also be used to derive plans for the waypoints using complex non-linear models based on historical data of similar routes in similar conditions e.g., season, days of the week, weather, etc. Perception algorithms can convert sensory input from cameras to recognise objects and other road participants, contributing to local scene understanding. It is common for these algorithms to be trained to detect, recognise, or track objects using supervised learning algorithms. Deep neural networks have become the first choice for object detection from cameras [14].

REFERENCES

[1] P. Oreizy, M. M. Gorlick, R. N. Taylor, D. Heimhigner, G. Johnson, N. Medvidovic, A. Quilici, D. S. Rosenblum, and A. L. Wolf, "An architecture-based approach to self-adaptive software," *IEEE Intelligent Systems and their Applications*, vol. 14, no. 3, pp. 54–62, 1999.

[2] D. Garlan, S.-W. Cheng, A.-C. Huang, B. Schmerl, and P. Steenkiste, "Rainbow: Architecture-based self-adaptation with reusable infrastructure," *Computer*, vol. 37, no. 10, p. 46–54, Oct. 2004. [Online]. Available: https://doi.org/10.1109/MC.2004.175

[3] M. Salehie and L. Tahvildari, "Self-adaptive software: Landscape and research challenges," *ACM Trans. Auton. Adapt. Syst.*, vol. 4, no. 2, May 2009.

[4] B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi, J. Magee, J. Andersson, B. Becker, N. Bencomo, Y. Brun, B. Cukic, G. Di Marzo Serugendo, S. Dustdar, A. Finkelstein, C. Gacek, K. Geihs, V. Grassi, G. Karsai, H. M. Kienle, J. Kramer, M. Litoiu, S. Malek, R. Mirandola, H. A. Müller, S. Park, M. Shaw, M. Tichy, M. Tivoli, D. Weyns, and J. Whittle, *Software Engineering for Self-Adaptive Systems: A Research Roadmap*. Springer, 2009, pp. 1–26.

[5] R. de Lemos, H. Giese, H. A. Müller, M. Shaw, J. Andersson, M. Litoiu, B. Schmerl, G. Tamura, N. M. Villegas, T. Vogel, D. Weyns, L. Baresi, B. Becker, N. Bencomo, Y. Brun, B. Cukic, R. Desmarais, S. Dustdar, G. Engels, K. Geihs, K. M. Göschka, A. Gorla, V. Grassi, P. Inverardi, G. Karsai, J. Kramer, A. Lopes, J. Magee, S. Malek, S. Mankovskii, R. Mirandola, J. Mylopoulos, O. Nierstrasz, M. Pezzè, C. Prehofer, W. Schäfer, R. Schlichting, D. B. Smith, J. P. Sousa, L. Tahvildari, K. Wong, and J. Wuttke, *Software Engineering for Self-Adaptive Systems: A Second Research Roadmap*. Springer, 2013.

[6] R. de Lemos, D. Garlan, C. Ghezzi, H. Giese, J. Andersson, M. Litoiu, B. Schmerl, D. Weyns, L. Baresi, N. Bencomo, Y. Brun, J. Camara, R. Calinescu, M. B. Cohen, A. Gorla, V. Grassi, L. Grunske, P. Inverardi, J.-M. Jezequel, S. Malek, R. Mirandola, M. Mori, H. A. Müller, R. Rouvoy, C. M. F. Rubira, E. Rutten, M. Shaw, G. Tamburrelli, G. Tamura, N. M. Villegas, T. Vogel, and F. Zambonelli, "Software engineering for self-adaptive systems: Research challenges in the provision of assurances," in *Software Engineering for Self-Adaptive Systems III. Assurances*, R. de Lemos, D. Garlan, C. Ghezzi, and H. Giese, Eds. Cham: Springer International Publishing, 2017, pp. 3–30.

[7] D. Weyns, *Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective*. Wiley, IEEE Press, 2020.

[8] J. Hellerstein *et al.*, *Feedback Control of Computing Systems*. USA: John Wiley Sons, Inc., 2004.

[9] A. Filieri *et al.*, "Control strategies for self-adaptive software systems," *ACM Trans. on Autonomous and Adaptive Systems*, vol. 11, no. 4, 2017.

[10] S. Shevtsov *et al.*, "Control-theoretical software adaptation: A systematic literature review," *IEEE TSE*, vol. 44, no. 8, pp. 784–810, Aug. 2018.

[11] N. Bencomo *et al.*, "RaM: Causally-Connected and Requirements-Aware Runtime Models using Bayesian Learning," in *Models*, 2019.

[12] P. Jamshidi *et al.*, "Fuzzy self-learning controllers for elasticity management in dynamic cloud architectures," in *QoSA*, April 2016.

[13] F. Quin, D. Weyns, T. Bamelis, S. S. Buttar, and S. Michiels, "Efficient analysis of large adaptation spaces in self-adaptive systems using machine learning," in *14th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. IEEE, 2019.

[14] J. Janai *et al.*, "Computer vision for autonomous vehicles: Problems, datasets and state-of-the-art," *arXiv preprint arXiv:1704.05519*, 2017.